Agentic AI with chatops for large scale network operations

1st Fatos Peci
Faculty of Electrical and Computer Engineering
University of Pristina
Pristina, Kosovo
fatos.peci@uni-pr.edu

2nd Enver Hamiti
Faculty of Electrical and Computer Engineering
University of Pristina
Pristina, Kosovo
enver.hamiti@uni-pr.edu

3rd Ishtiaq Khan
Equinix Inc
Santa Clara, USA
iskhan@equinix.com

2025 IEEE Conference on Artificial Intelligence (CAI) | 979-8-3315-2400-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/CAI64502.2025.00242

***Abstract*— Operating large-scale networks necessitates interaction with numerous tools and dashboards for troubleshooting and analysis. Given the sensitivity of network services, minimizing manual interaction time is crucial for improving Mean Time To Repair (MTTR) and ensuring service availability. Recent developments in generative AI enable the transformation of interactions with diverse APIs and dashboards into intent-driven operations. Agentic AI offers capabilities to translate user intent into automated processes, thereby reducing the time engineers spend interacting with various interfaces and dashboards. This paper presents a novel approach for operating large-scale network infrastructure, wherein AI agents execute tasks involving API and dashboard interactions based on natural language commands. We evaluated the impact of this approach within a major global network environment. Our findings demonstrate significant time savings, consequently improving MTTR and overall service availability in this large-scale network.**

Keywords—service level agreements, mean time to repair, natural language processing, large language models, chatops

I. Introduction

Operating large-scale networks has traditionally involved significant manual intervention. Every manual task is prone to human error although these can be repetitive tasks. These tasks can be categorized into two large groups: provisioning and troubleshooting. Deployment tasks have very high frequency since it is driven by the speed of customer requirements. Customer requirements are directly linked to the speed their business grows. Mainly customers in these large scale networks are cloud service providers, tier2 internet service providers and other top tier telcos. Amount of provisioning tasks can considerably change throughout time of day, timezones or events. Thus planning the resources to conduct these provisioning tasks can be difficult. On the other hand the troubleshooting tasks are dependent directly on the amount of provisioning tasks and other external factors such as third party carriers, subsea cable cuts, network regular maintenance, old and faulty equipment, improper corrective actions.

The diverse systems supporting monitoring, tracking, locating, storing, and digital interaction within network infrastructure often come from various software suppliers; some provide modern APIs, while others are legacy tools. Keeping a single pane of glass for these tasks can be daunting for engineers. Requiring engineers to interact with multiple, disparate tools for various purposes often increases delays in task completion and service restoration.

Recent developments in generative AI have given a new momentum in tool development. Specifically, generative AI's ability to translate human intent into tool interactions can significantly reduce the operational effort compared to traditional methods. Another significant factor driving adoption is the ease with which these AI-driven tools can be built. A large ecosystem of frameworks, both open-source and commercial, already exists to support the development of automated workflows. Notable open-source frameworks, including Langchain and LangGraph (utilized in this work), facilitate such development. These tools enable Retrieval-Augmented Generation (RAG), connect to vector databases, and integrate Large Language Models (LLMs) and embedding models into a single ecosystem; LangGraph specifically provides toolsets for creating agentic AI systems. This new agentic AI toolset provides a single interface to interact with diverse underlying tools, enabling a plethora of workflows within the network operations environment.

Integrating AI agents with sensitive data environments needs to be secure, as they can directly interact with services during both provisioning and troubleshooting phases. The proliferation of hardware (e.g., NVIDIA DGX) and software (e.g., Ollama) supporting local model execution has enabled private AI solutions where confidential information can be processed by the agentic environment without leaving the local network premises.

Evaluation frameworks and guardrails constitute the final step in building an end-to-end, production-grade solution. To enable observability and evaluation we have chosen open-

979-8-3315-2400-5/25/$31.00 ©2025 IEEE
DOI 10.1109/CAI64502.2025.00242

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:28 UTC from IEEE Xplore. Restrictions apply.

source solution called truelens. This framework enhances observability by providing metrics such as groundedness, answer relevance, context relevance, latency, and PII. This enables production-grade applications to adhere to responsible AI principles, including explainability, fairness, robustness, transparency, and privacy.

In conclusion, network operations represent an exemplary use case demonstrating how agentic AI can significantly improve workflows and increase velocity in provisioning and troubleshooting tasks. In the following section we will discuss some of the related work in the field of agentic AI, then move on to explain in particular how this novel approach can be applied in operation inside particular network environment and finally present recorded results demonstrating the impact of this transformation in day-to-day operations in time savings as well as availability increase of services.

II. Related Work

Research in the field of agentic AI has recently garnered increased interest, with frameworks such as ReAct, Reflextion and Refine [1] [2] [3]. Each framework employs a distinct method for improving basic LLM capabilities, but fundamentally, they utilize feedback system to iteratively improve upon previous generation without any additional training or reinforcement learning. Relying solely on reflective abilities has improved performance across many diverse tasks, ranging from dialog response, code optimization, constrained generation and mathematical reasoning.

Another design pattern in agentic development is ability to use external tools, to augment generation for purpose of increasing relevancy in the data out of the training set. Network environments contain numerous tools, and the agent's ability to accommodate a large number of these tools impacts system design. In this regard, a significant amount of research has been conducted over the past year such as: Gorilla, Toolformer and MM-ReAct [4][5][6], involving tool calling and response generation.

Another pattern in agentic AI applications is planning, the process by which agents autonomously devise a sequence of steps to complete a larger task. Closely related to this concept is multi-tool usage and deciding upon which trajectory to use in tool calling. Most of the research categorizes planning tasks in the following sets: task decomposition, multi-plan selection, memory-aided planning and external-planner aided [7].

The final pattern observed in agentic AI is multi-agent collaboration, which stems from the principle that a single agent cannot do all the tasks qualitatively whereas a purpose-built agent which is given a specialized role using possibly even a different LLM can perform better the task under consideration. It has been shown [8] that each agent by having a specific role, workflow and memory can provide better results. Major players exist in the field of multi-agent frameworks, including the open-source AutoGen, CrewAI, and LangGraph [9][10][11], which facilitate the development of multi-agent applications.

In network operations, besides workflows, there are significant information retrieval tasks, particularly during troubleshooting. Retrieval-Augmented Generation (RAG) [12] has shown promising results in retrieving necessary context from multiple data sources. One use case in operations is to retrieve past incidents, historical knowledge, and various relevant sources for troubleshooting the existing issue. Significant research has been conducted in recent years to improve RAG workflows, which are primarily categorized as naive, advanced, and modular [13].

Naïve RAG have three building blocks which are: indexing, retrieval, and generation from frozen LLM. Typically, all RAG architectures involve several steps as mentioned in [14] which are: chunk optimization, fine-tuning embedding models, query rewriting, fine-tuning retrievers, embedding transformation, information compression, and re-ranking.

In order to measure and evaluate RAG metrics such as: Retrieval Quality (Hit Rate, MRR, and NDCG) and generation Quality (faithfulness, relevance, and non-harmfulness) [15] different automatic evaluation frameworks have been developed, like RAGAS in [16], ARES [17] and TruLens [18].

While RAG improves LLM capabilities by providing relevant and up-to-date information, it faces different challenges such as context length, which depends on the context window size of LLM [19], and robustness when handling conflicting input [20].

While current research in field of agentic AI and RAG offers great environment for enabling these capabilities in network operations, they are not 'plug-and-play'; significant development and design effort is required to customize them for a particular industry vertical. The following section presents a specific use case and design to enable these capabilities in network operations.

III. Architecture and Experimental Work

ChatOps has been recently introduced as terminology in networking world due to the increased use of natural language processing (NLP) advancements [21]. The advent of LLMs has facilitated the understanding of both basic networking concepts and human queries [22]. We aim to provide a single interface allowing engineers to assign provisioning or troubleshooting tasks to an agent; these tasks are then tracked and resolved by combining API data from various tools with planning and reasoning capabilities.

Figure 1. shows the overall architecture where the query is received via a chat interface which could be any enterprise collaboration platform (teams, slack, jabber etc.) acting as frontend where all network teams have access to and track and escalate their ticket and tasks. A WebSocket connection is established with the agent, which listens for mentions, extracts intent, and performs appropriate planning and tool selection [23].

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:28 UTC from IEEE Xplore. Restrictions apply.

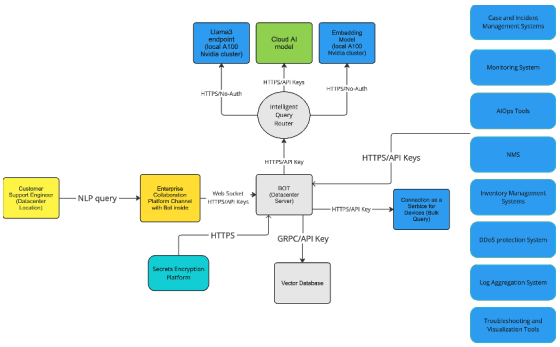


Fig. 1. Flow diagram of chatops for network operation

LLM is carefully selected with function calling capabilities [24].

Local LLM(GPU-equipped HW)	Public LLM (for inference only)
mistral-small:24b-instruct-2501-fp16	*GPT-3.5-Turbo*

The available toolset plays a critical role in the bot's capabilities, as the bot must interface with APIs to provide context for the query [29]. In our experimental environment available tools include: case and incident management systems, monitoring system, AIOps tools, NMS, Inventory Management Systems, DDoS protection systems, Log Aggregation Systems, Troubleshooting and Visualization Toolkits.

A. Spare Locator

A primary issue affecting network health is node failure which can be transient, usually the remedial is done via change such as a software upgrade, however, in large networks, the probability of encountering faulty hardware is higher. In such cases, the usual remediation is via replacement, finding a spare device or determining its availability is crucial when customer services are impacted [22]. Spare is not always available in the same physical location as affected part and the engineer must locate it in the nearest network location or find a similar device that supports similar functionality. These tasks consume considerable engineer time, to improve this operation, we have offered an approach to interface via API and query the bot via NLP query e.g. spare part for device A (Figure 2).

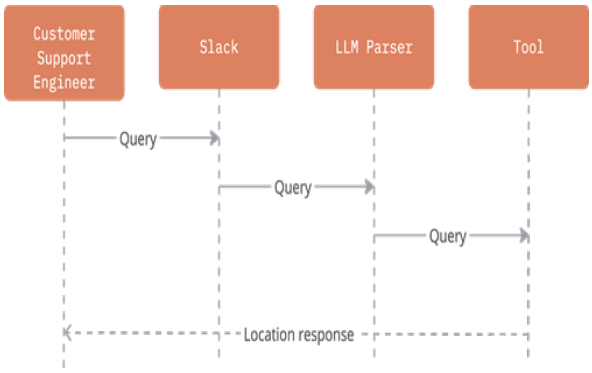


Fig. 2. Flow diagram of messaging in chatops

Knowing the location of device A is important and then find spares in similar or close location to affected device, in addition, we also need to find devices that are in store with compatible capabilities. In this particular case, the LLM needs device location information, device capabilities and geographical location, which could be stored in netbox as it contains all network inventory. We create a query to list affected device network location, capabilities and geoinformation via API then, upon receiving the response, we instruct the LLM to find similar information within the embedded database which has been created from the inventory database (Figure 2) [12]. A comparison between manual operation and the automated approach using an LLM yielded the metrics shown in Figure 3.

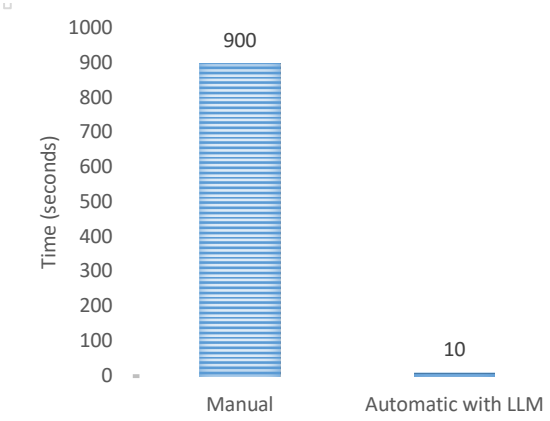


Fig. 3. Comparison of manual vs automatic approach

B. Console Info

Furthermore, verifying the operational status of spare devices during incidents is critically important. These devices are typically new, unconfigured, and accessed via out-of-band console connections. Previously, managing this information within SharePoint or Excel formats proved inefficient for timely data retrieval and maintenance. Consequently, a database accessible via an API was established as the authoritative source ('source of truth') for all console-related data. This system

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:28 UTC from IEEE Xplore. Restrictions apply.

involves segmenting ('chunking') and embedding the console data into a vector database, which facilitates engineer-initiated queries for searching and retrieving necessary information [33]. The interaction between different elements is shown in Figure 4.

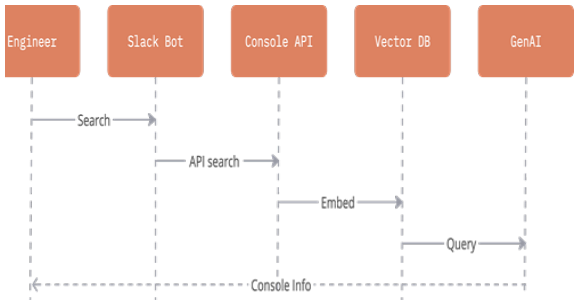


Fig. 4. Flow diagram for console info search

The total time saved by using the intelligent method in retrieving these results in our experiments is shown in Figure 5.

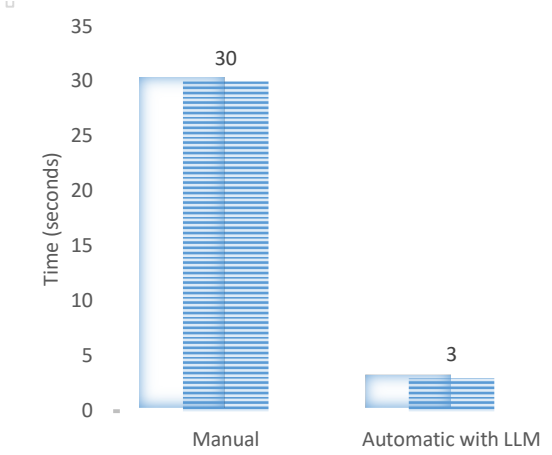


Fig. 5. Console search time savings

C. Packet loss and latency

A major cause of incidents during network operations is latency and packet loss. There are many causes of latency and packet loss: dust collected on Small Factor Pluggable (SFP), upstream providers having fiber cuts, interface errors, inefficient routing, issues in the control plane, software bugs, device incompatibility, wrong configuration, maintenance [22]. Locating packet loss and latency is of paramount importance in large network environments where you have multiple vendors and transit providers for interconnectivity. Global-scale network services can traverse subsea cables, multiple vendors and country borders thus it is crucial to locate and find the cause of the issue very quickly. Readily available monitoring tools for detecting packet loss or latency often include traceroute and ping utilities. For global networks, we have usually ping traces running constantly within the network between different network locations. In our experiments, while an API could be queried for packet loss or latency data, the raw output is not directly consumable by an engineer (Figure 6). We prompt a large language model to transform JSON output, retrieve the necessary data along with temporal information and present it in human-readable format.

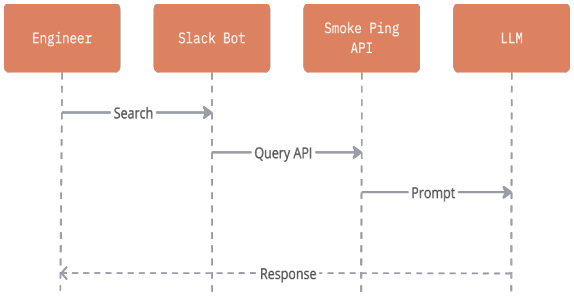


Fig. 6. Flow Diagram for packet loss and latency

The time required to manually retrieve and process this information is considerable, especially given the sensitivity of services experiencing loss and latency. In Figure 7, we have shown metrics of time it is taking for tasks via the manual method vs automatic with LLM.

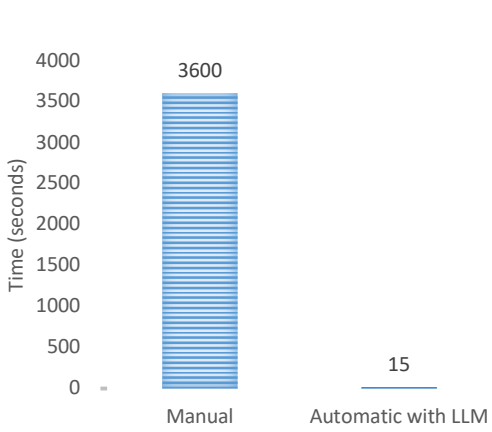


Fig. 7. Packet loss and latency retrieval

D. Path retrieval

During major incidents, network control and data plane information plays a crucial role in diagnosing and resolving issues. For many routing protocols used in service providers, control plane reachability can be retrieved from a single node by way of a link-state database (LSDB). LSDB shows adjacency changes, neighbor information, k-shortest paths, metrics towards different destinations, redundant paths, router isolations, path changes, flaps. Tools offer visibility into control and data plane by providing API access to LSDB. Using large language models improves interaction between lsdb and engineer creating a translation layer which facilitates diagnosing issue by giving clear picture at each specific point in time. In Figure 8 we show interaction between different elements in our

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:28 UTC from IEEE Xplore. Restrictions apply.

experiments where LLM offers translation between information from route explorer tools towards engineer.

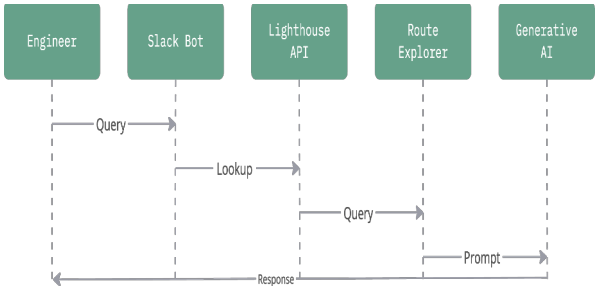


Fig. 8. Path retrieval with LLM

On the other hand manual process involves searching for mpls label and lookup inside each device to track all the available paths. This cumbersome manual process is replaced by an automated one that utilizes an API call to retrieve path information, thus immensely reducing the time to get all the available paths. Since LSDB history is stored for several months, it can also be used to troubleshoot past issues, a capability not feasible with other procedures. Overall time saved for the path retrieval process is shown in Figure 9.

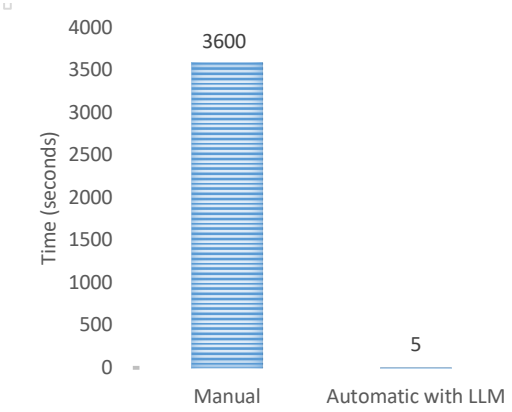


Fig. 9. Available path lookup

E. Node isolation

As network maintenance for patches and fixes is unavoidable, it represents a primary cause of network outages. Thus, the ability to detect any impact before, during, or after maintenance is of paramount importance. Many critical incidents result from improper router isolation processes, which are typically employed during maintenance to prevent service impact. In addition to detecting isolation events, the notification process is critical; this can be achieved by paging engineers, enabling them to promptly address these critical events with the necessary priority and urgency. Figure 10 depicts the messaging flow between different backend systems offering engineers with proper alerting in case of events.

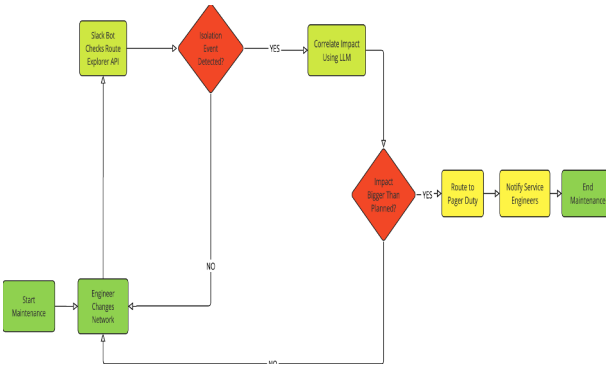


Fig. 10. Node isolation flow chart

Employing this flow during our experiments tremendously reduced the reaction time to potential outage-causing events. In Figure 11, we show the average number of seconds saved by using our approach.

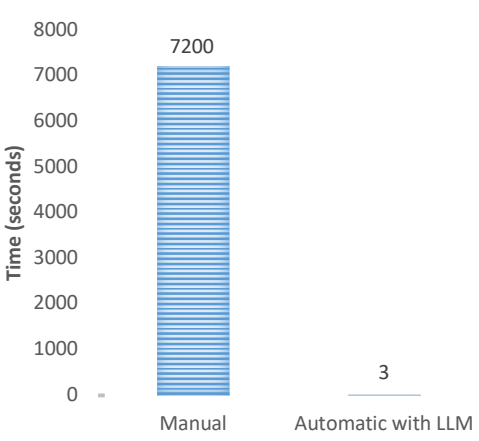


Fig. 11. Router isolation detection via AI

F. Vendor search

Utilizing multiple device vendors can reduce Operating Expenditures (OpEx) and enhance reliability by isolating software issues to specific vendor equipment. A disadvantage, however, is increased troubleshooting complexity, as engineers often specialize in particular vendors. Furthermore, it is difficult to isolate interoperability issues. Through the operation of their devices across diverse enterprise and service provider environments, vendors have developed extensive knowledge bases regarding products and best practices, constituting valuable troubleshooting resources. Vendors often create specific signatures or problem articles corresponding to known issues. Providing engineers with ready access to these articles, triggered by specific log events, can supply information crucial for identifying root causes and ultimately resolving issues. Consequently, we devised the approach shown in Fig. 12: upon an event detected by the correlation tool, a retrieval-augmented

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:28 UTC from IEEE Xplore. Restrictions apply.

search is initiated across logs from the relevant timeline for each vendor. This search attempts to find pertinent information within vendor knowledge bases, with the results summarized using AI.

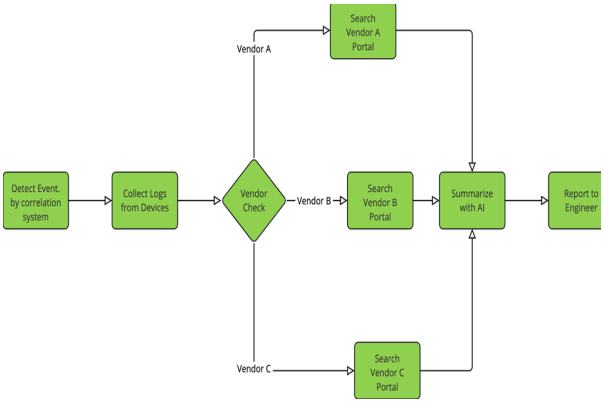


Fig. 12. Vendor search using AI

An important metric for evaluating this approach is the time required to research vendor documentation or portal responses that contributed to issue detection and rectification (Figure 13).

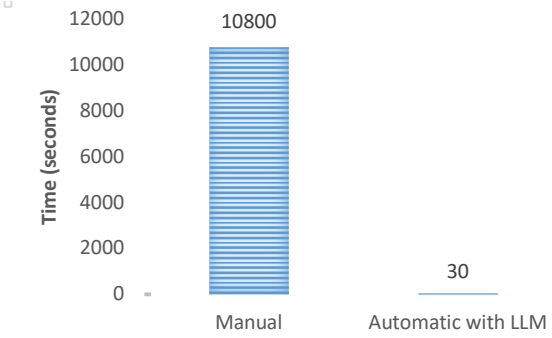


Fig. 13. Vendor Search using AI

G. Backbone links flapping event and remedy

It is normal occurrence in large scale network networks spanning across regions(local and global) to have network backbone link flaps - a situation that, when handled manually demands a series of time-consuming steps. Traditionally, a network engineer would need to manually review alerts, login to various systems, run diagnostic verifications or simulations manually, update configuration files, and then execute configuration changes, a process that can easily take around ~30 minutes per event. Not only is this approach slow, but it also increases the risk of human error, which can lead to network downtime or misconfigurations

The total time saved by using the intelligent method from detection, simulation, validation, taking calculated action and finally reporting for remedy is key metric.

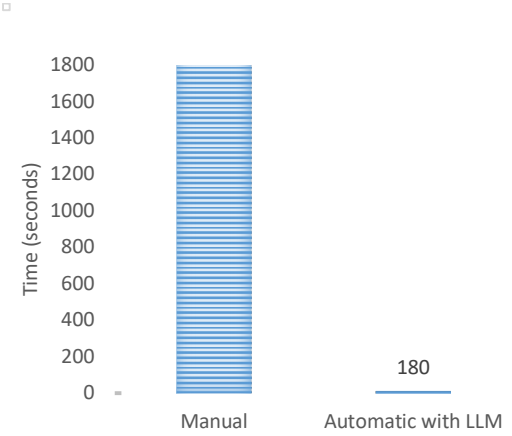


Fig. 14. Flapping BB links remedy via AI

This is not an easy problem to automate as provided mechanism of existing network protocols do not have intelligence to consider isolation (control-plane) when dampening the link based on flaps. In contrast, an automated workflow that leverages a map (LinkEntity) integrating control and data plane information, triggered via an intelligent agentic system, dramatically reduces this response time. In our design, once flapping events are detected via Kafka (across multiple topics), they are mapped and aggregated to a LinkEntity (representing control and data-plane state). The observability system then automatically verifies if a predefined threshold (e.g., three or more flaps within 60 seconds) is met; The system queries the Bot with event details (source and target nodes). The Bot subsequently performs pre-checks using digital twin simulations and Shortest Path First (SPF) path tracing (considering Equal-Cost Multi-Path, ECMP). It first simulates modifying the metric value of the impacted link and then re-checks the simulation to confirm successful failover to a backup link; all performed without manual intervention via REST API calls to the observability tool, which exposes available multi-path information and source/target node conditions (e.g., whether a node has its overload bit set); Based on the received data indicating success and backup link availability (e.g., status: success, payload: backup link available), the Generative AI invokes a function (via REST API) targeting the central configuration management system. This function call includes the source, target, and a new metric (e.g., static: 20000) to trigger the configuration update and push the change. As a result, the entire remediation process—from alert detection to configuration push and notification—is accomplished in approximately 3 minutes.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:28 UTC from IEEE Xplore. Restrictions apply.

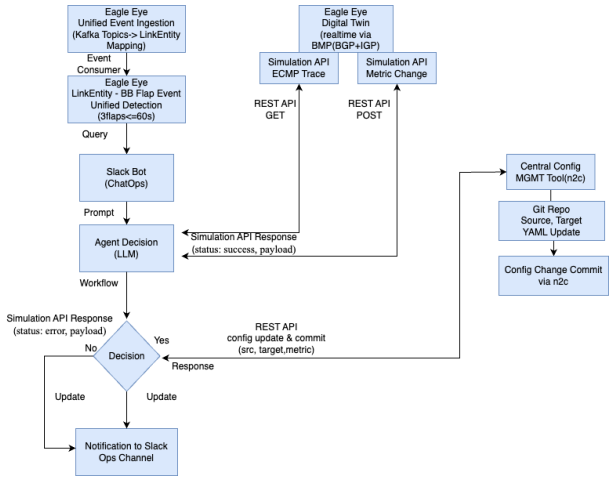


Fig. 15. Link flaps remedy workflow

Our novel approach integrates data plane information (specifically, SNMP trap-based events for backbone interface flaps consumed via an observability application through a Kafka topic) [34] with real-time control-plane visibility obtained via BGP Monitoring Protocol (BMP, covering BGP and IGP) and BGP Address Family Identifiers (AFIs) [35]. This combined data streams in real-time via specific Kafka topics, is consumed, and mapped to a unified LinkEntity datastore. Workflow actions are triggered based on matching predefined thresholds, such as:

- Backbone link flap event trigger: Ingests real-time alerts (via Kafka topics), mapped to unified LinkEntity (datastore) that maps control and data plane events; the observability application triggers an event notification to the Bot, initiating the workflow for handling flap events exceeding the threshold, providing the source and target nodes.

- Agent decision engine: A local LLM–powered agent that interprets events (e.g. link flapping detected when >=3 up/down events in <60s) and orchestrates the workflow for simulation first

- Simulation w/ digital twin: Execute pre-checks using network REST API for Source, Target node (via SPF) to simulate real-time alternate paths and validate that changes will not isolate nodes; the NetworkX computes ECMP and gives all Nodes and Edges (to-from nodes, w/ weight as igp_metric) to ensure there is no isolation expected.
- Configuration management & orchestration: Config update REST API is triggered as part of workflow to config management orchestrator that manages network config stored as YAML files in Git and pushes verified changes w/ feedback response (status: success and payload)
- Notification & reporting: Updates operations teams (via static #slack-channel) with details of change and time of commit. This design not only addresses the immediate challenge of backbone link flapping but also sets the stage for a broader, agentic approach to network operations in dynamic SP environments unpredicted simulation inaccuracies
- Future Enhancements: Robust rollback mechanisms: Enhance rollback protocols with automated failover in case of issues post-change. Extended integration: Support additional observability sources and simulation models to cover a wider range of network scenarios

H. ChatOps multi-tool aggregator workflow

For our use-cases we established pre-defined workflow for Network Ops Teams when they escalate to our group via Slack we leveraged ChatOps (Slack Bot) to trigger the workflow based on required parameters (Unique ID) in our use-case. Once SRE Bot observes the thread w/ tag and UUID we run pre-define work flow based on multi-tool query for same end customer, reason that via LLM and give Ops the valuable insight of what is related to specific customer. This aggregator use many of the use-cases we have defined above and drastically narrows down the Mean Time To Repair (MTTR) exponentially by having low-level data available on the same thread without user going to multiple dashboard and doing the queries manually.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:28 UTC from IEEE Xplore. Restrictions apply.

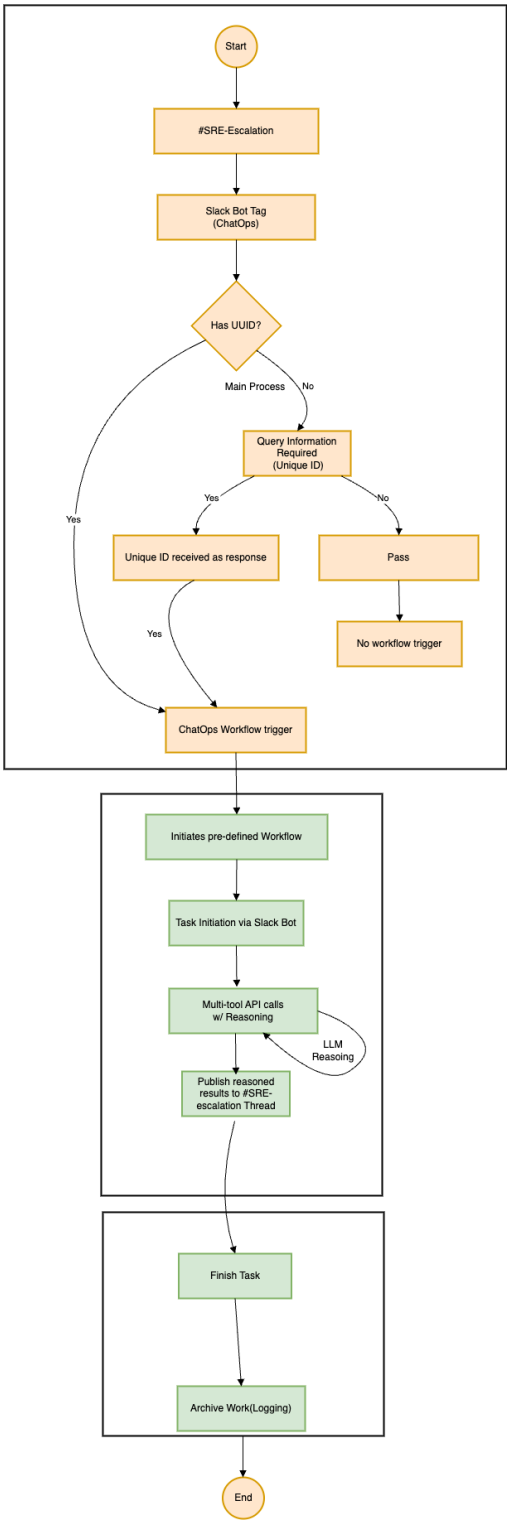


Fig. 16. Flapping BB links remedy using AI

IV. RESULTS

To present the results of this research, we collected operational data over several months from a major network provider to determine the frequency of the tasks discussed in this article. This data allows us to measure the impact of our proposed approach on Mean Time To Repair (MTTR) and overall network availability. In Table 1. we have shown the average frequency of each task.

TABLE I. FREQUENCY OF TASKS PER WEEK

Tasks	**Spare device search**	**Console info**	**Packet loss and Lat.**	**Search for path**	**Node isolation**	**Vendor search**
Frequency per week	10	20	60	60	6	60

Another valuable comparison highlights the time saved (in seconds) when utilizing automation with LLMs versus the traditional manual approach. Table 2 presents this comparison, offering a compelling case for adopting LLM-based automation in daily operations. The calculated weekly MTTR savings per task achieved through this automation range significantly, from approximately 8 minutes to over 100 hours.

TABLE II. COMPARISON OF APPROACH VS TASKS

Tasks	**Spare device search**	**Console Info**	**Packet loss and Lat.**	**Search for path**	**Node isolation**	**Vendor search**
Frequency of task per week	10	20	60	60	6	60
Manual (sec)	9000	600	216000	216000	43200	648000
Automatic with LLM (sec.)	100	100	900	300	18	1800

The time saved directly impacts service availability and, consequently, Service Level Agreements (SLAs), as Availability is inversely related to MTTR, as shown in formula (1)

$$Availability = \frac{MTBF}{MTBF+MTTR} \quad (1)$$

MTBF - represents Mean Time Between Failures, a measure of system or device reliability, which typically cannot be directly influenced by operational procedures.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:28 UTC from IEEE Xplore. Restrictions apply.

To calculate per-service Availability, we will take for example an existing layer 2 production service connecting two major locations London to Singapore (Figure 14).

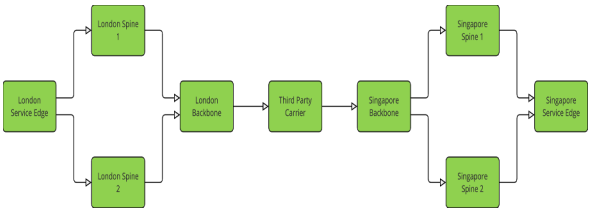


Fig. 17. Service from London to Singapore

$$Total\ Avail.per\ service\ (excl.\ links) = Avail_{Lond\ SE} \times (1 - (1 - Avail_{LondSP1}) \times (1 - Avail_{LondSP2})) \times Avail_{LondBB} \times Avail_{SingBB} \times (1 - (1 - Avail_{SingSP1}) \times (1 - Avail_{SingSP2})) \times Avail_{SingSE}$$

(2)

Formula (2) demonstrates how the availability of each component along the service path directly impacts the overall end-to-end service availability. Consequently, service availability is inversely dependent on MTTR, which, as demonstrated, is directly impacted by the time taken for the aforementioned operational tasks (spare search, console information retrieval, packet loss/latency analysis, path searching, node isolation verification, and vendor information search). Therefore, overall service availability can be significantly improved by reducing the time engineers spend performing these essential tasks.

V. Conclusions

Emergent capabilities on large language models such as in-context learning have demonstrated significant potential across various research fields. We have explored this potential within the specific context of service operations in large-scale network operations environments. Our results demonstrate that by integrating predictive elements (such as static coding or predefined workflows) with Generative AI, it is possible to bridge the operational gap between complex API-driven systems and engineers, significantly reducing MTTR and thereby enhancing the overall availability of these intricate systems. Future research could focus on integrating these diverse capabilities as distinct tools accessible via function calling within an LLM framework. This would enable the LLM to contextually determine the appropriate API or combination of APIs required for complex, multi-step tasks. Furthermore, incorporating methods for chat history compression and message importance re-ranking could allow for maintaining relevant conversational context within the LLM's input constraints.

References

[1] Y. J. Kim, R. Henry, R. Fahim, and H. H. Awadalla, "FineQuant: Unlocking Efficiency with Fine-Grained Weight-Only Quantization for LLMs," Aug. 2023, [Online]. Available: http://arxiv.org/abs/2308.09723

[2] A. Maatouk, N. Piovesan, F. Ayed, A. De Domenico, and M. Debbah, "Large Language Models for Telecom: Forthcoming Impact on the Industry," Aug. 2023, [Online]. Available: http://arxiv.org/abs/2308.06013

[3] Z. Wang et al., "RCAgent: Cloud Root Cause Analysis by Autonomous Agents with Tool-Augmented Large Language Models," Oct. 2023, [Online]. Available: http://arxiv.org/abs/2310.16340

[4] D. Zhang, X. Zhang, C. Bansal, P. Las-Casas, R. Fonseca, and S. Rajmohan, "PACE-LM: Prompting and Augmentation for Calibrated Confidence Estimation with GPT-4 in Cloud Incident Root Cause Analysis," Sep. 2023, [Online]. Available: http://arxiv.org/abs/2309.05833

[5] E. Almazrouei et al., "The Falcon Series of Open Language Models," Nov. 2023, [Online]. Available: http://arxiv.org/abs/2311.16867

[6] T. Ali and P. Kostakos, "HuntGPT: Integrating Machine Learning-Based Anomaly Detection and Explainable AI with Large Language Models (LLMs)," Sep. 2023, [Online]. Available: http://arxiv.org/abs/2309.16021

[7] T. Wittkopp, A. Acker, and O. Kao, "Progressing from Anomaly Detection to Automated Log Labeling and Pioneering Root Cause Analysis," Dec. 2023, [Online]. Available: http://arxiv.org/abs/2312.14748

[8] A. Maatouk, F. Ayed, N. Piovesan, A. De Domenico, M. Debbah, and Z.-Q. Luo, "TeleQnA: A Benchmark Dataset to Assess Large Language Models Telecommunications Knowledge," Oct. 2023, [Online]. Available: http://arxiv.org/abs/2310.15051

[9] S. Zhang et al., "OPT: Open Pre-trained Transformer Language Models," May 2022, [Online]. Available: http://arxiv.org/abs/2205.01068

[10] B. Workshop et al., "BLOOM: A 176B-Parameter Open-Access Multilingual Language Model," Nov. 2022, [Online]. Available: http://arxiv.org/abs/2211.05100

[11] G. Penedo et al., "The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only," Jun. 2023, [Online]. Available: http://arxiv.org/abs/2306.01116

[12] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," Dec. 2023, [Online]. Available: http://arxiv.org/abs/2312.10997

[13] R. Zhao et al., "Retrieving Multimodal Information for Augmented Generation: A Survey," Mar. 2023, [Online]. Available: http://arxiv.org/abs/2303.10868

[14] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," Sep. 2023, [Online]. Available: http://arxiv.org/abs/2309.15217

[15] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," Nov. 2023, [Online]. Available: http://arxiv.org/abs/2311.09476

[16] C. Packer et al., "MemGPT: Towards LLMs as Operating Systems," Oct. 2023, [Online]. Available: http://arxiv.org/abs/2310.08560

[17] W. Yu, H. Zhang, X. Pan, K. Ma, H. Wang, and D. Yu, "Chain-of-Note: Enhancing Robustness in Retrieval-Augmented Language Models," Nov. 2023, [Online]. Available: http://arxiv.org/abs/2311.09210

[18] B. Wang et al., "Shall We Pretrain Autoregressive Language Models with Retrieval? A Comprehensive Study," Apr. 2023, [Online]. Available: http://arxiv.org/abs/2304.06762

[19] U. Alon, F. F. Xu, J. He, S. Sengupta, D. Roth, and G. Neubig, "Neuro-Symbolic Language Modeling with Automaton-augmented Retrieval," Jan. 2022, [Online]. Available: http://arxiv.org/abs/2201.12431

[20] M. A. Ferrag, A. Battah, N. Tihanyi, M. Debbah, T. Lestable, and L. C. Cordeiro, "SecureFalcon: The Next Cyber Reasoning System for Cyber Security," Jul. 2023, [Online]. Available: http://arxiv.org/abs/2307.06616

[21] J. Bandlamudi et al., "Towards Hybrid Automation by Bootstrapping Conversational Interfaces for IT Operation Tasks," 2023. [Online]. Available: https://www.redhat.com/en/technologies/management/ansible

[22] J. Wulf and J. Meierhofer, "EXPLORING THE POTENTIAL OF LARGE LANGUAGE MODELS FOR AUTOMATION IN TECHNICAL CUSTOMER SERVICE," 2024.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:28 UTC from IEEE Xplore. Restrictions apply.

[23] J. Wang et al., “Network Meets ChatGPT: Intent Autonomous Management, Control and Operation,” 2023.

[24] K. An et al., “Nissist: An Incident Mitigation Copilot based on Troubleshooting Guides,” Feb. 2024, [Online]. Available: http://arxiv.org/abs/2402.17531

[25] Y. Jiang et al., “Xpert: Empowering Incident Management with Query Recommendations via Large Language Models,” Dec. 2023, [Online]. Available: http://arxiv.org/abs/2312.11988

[26] D. Wu et al., “NetLLM: Adapting Large Language Models for Networking,” Feb. 2024, doi: 10.1145/3651890.3672268.

[27] Y. Huang et al., “Large Language Models for Networking: Applications, Enabling Techniques, and Challenges,” Nov. 2023, [Online]. Available: http://arxiv.org/abs/2311.17474

[28] D. Roy et al., “Exploring LLM-based Agents for Root Cause Analysis,” 2017. doi: XXXXXXX.XXXXXXX.

[29] M. Shetty, C. Bansal, S. P. Upadhyayula, A. Radhakrishna, and A. Gupta, “AutoTSG: Learning and Synthesis for Incident Troubleshooting,” May 2022, [Online]. Available: http://arxiv.org/abs/2205.13457

[30] Netboxlabs, “The Premier Network Source of Truth,” Netbox. Accessed: Aug. 16, 2024. [Online]. Available: https://netboxlabs.com/docs/netbox/en/stable/

[31] Ciena, “Ciena Blueplanet ROA Route explorer,” Ciena Blueplanet ROA Route explorer. Accessed: Aug. 16, 2024. [Online]. Available: http://www.blueplanet.com/products/roa-route-explorer.html

[32] Selector AI, “Get end-to-end visibility and contextual insights for your complex hybrid networks,” Selector AI. Accessed: Aug. 16, 2024. [Online]. Available: https://www.selector.ai/solutions/network-connectivity/

[33] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” Oct. 2022, [Online]. Available: http://arxiv.org/abs/2210.03629

[34] SNMP, https://datatracker.ietf.org/doc/html/rfc1157#section-4.1.6.3

[35] BMP, https://datatracker.ietf.org/doc/html/rfc7854#page-18

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:28 UTC from IEEE Xplore. Restrictions apply.